Модуль 1

Введение в Node.js

Что такое Node.js • V8 • Событийный цикл • Установка • nvm • npm • Первый скрипт • REPL

1.1 Что такое Node.js

Node.js — это среда выполнения (runtime) JavaScript вне браузера. До появления Node.js в 2009 году JavaScript мог работать только внутри браузера: управлять DOM-деревом, обрабатывать клики и анимации. Райан Дал (Ryan Dahl) взял движок V8 из Google Chrome, добавил к нему API для работы с файловой системой, сетью и операционной системой — и получился Node.js.

Движок V8 компилирует JavaScript в машинный код, что обеспечивает высокую скорость выполнения. Node.js не является ни языком программирования, ни фреймворком — это именно среда, в которой выполняется JavaScript-код на сервере.

Ключевые особенности Node.js:

- Однопоточность с событийным циклом (Event Loop) — вместо создания нового потока на каждый запрос Node.js использует один поток и очередь событий, что экономит память
- Неблокирующий ввод/вывод (Non-blocking I/O) — операции чтения файлов, запросы к БД и сети не блокируют основной поток, а выполняются в фоне
- Высокая производительность для I/O-задач — Node.js идеален для приложений с большим количеством одновременных подключений (чаты, API, стриминг)
- Огромная экосистема — npm (Node Package Manager) содержит более 2 миллионов пакетов
- Один язык для фронтенда и бэкенда — команда может использовать JavaScript везде, что упрощает разработку

Где используется Node.js на практике:

- REST API и GraphQL серверы — основное применение, обработка HTTP-запросов
- Реалтайм-приложения — чаты, онлайн-игры, совместное редактирование (WebSocket)
- Микросервисы — лёгкие сервисы, которые быстро стартуют и потребляют мало памяти

- CLI-утилиты — webpack, vite, eslint, prettier написаны на Node.js
- SSR (Server-Side Rendering) — Next.js, Nuxt.js рендерят страницы на сервере
- Десктоп-приложения — VS Code, Slack, Discord построены на Electron (Node.js + Chromium)

Важно понимать, для чего Node.js НЕ подходит: тяжёлые вычисления (обработка видео, машинное обучение, сложные математические расчёты). Поскольку Node.js однопоточный, длительные CPU-операции блокируют весь сервер. Для таких задач лучше использовать Python, Go или Rust.

1.2 Установка Node.js

Существует несколько способов установки Node.js. Мы рекомендуем использовать менеджер версий nvm, потому что он позволяет легко переключаться между версиями Node.js. Это важно, когда разные проекты требуют разных версий.

Способ 1: nvm (рекомендуемый)

nvm (Node Version Manager) — утилита для управления версиями Node.js. С ней вы можете установить несколько версий одновременно и переключаться между ними одной командой.

Установка nvm на macOS / Linux:

```
# Установка nvm
curl -o- https://raw.githubusercontent.com/nvm-
sh/nvm/v0.39.7/install.sh | bash


# Перезапустите терминал, затем:
nvm install --lts          # Установить LTS-версию (рекомендуется)
nvm install 20             # Установить конкретную версию
nvm use 20                 # Переключиться на версию 20
nvm ls                     # Список установленных версий
nvm alias default 20       # Установить версию по умолчанию
```

LTS (Long Term Support) — версия с долгосрочной поддержкой. Она получает исправления безопасности в течение 30 месяцев. Для продакшена всегда используйте LTS.

Установка на Windows:

На Windows используется отдельный проект nvm-windows. Скачайте установщик с github.com/coreybutler/nvm-windows и выполните:

```
nvm install lts
nvm use lts
```

Проверка установки:

```
node --version    # v20.x.x
npm --version     # 10.x.x
```

Если обе команды выводят номера версий — установка прошла успешно. Вместе с Node.js автоматически устанавливается npm — менеджер пакетов, о котором мы подробно поговорим в модуле 2.

1.3 Глобальный объект process

В браузере глобальным объектом является window. В Node.js его аналог — объект process. Он предоставляет информацию о текущем процессе и позволяет взаимодействовать с операционной системой.

Основные свойства и методы process:

- process.version — версия Node.js (например, "v20.11.0")
- process.platform — операционная система ("darwin" для macOS, "linux", "win32")
- process.cwd() — текущая рабочая директория (откуда запущен скрипт)
- process.argv — массив аргументов командной строки
- process.env — переменные окружения (например, process.env.HOME, process.env.PATH)
- process.exit(code) — завершить процесс (0 = успех, 1 = ошибка)
- process.memoryUsage() — использование памяти
- process.uptime() — время работы процесса в секундах

Создайте файл app.js:

```
// app.js
console.log('Привет, Node.js!');
console.log('Версия Node:', process.version);
console.log('Платформа:', process.platform);
```

```
console.log('Текущая директория:', process.cwd());
console.log('Аргументы:', process.argv.slice(2));
console.log('Память:', process.memoryUsage().heapUsed, 'байт');
```

Обратите внимание на process.argv.slice(2): первые два элемента массива argv — это путь к Node.js и путь к скрипту. Пользовательские аргументы начинаются с индекса 2.

Для запуска скрипта, выполните команду

```
node app.js
```

Для запуска скрипта и передачи в него аргументов, выполните

```
node app.js --name Alex --port 3000
```

1.4 REPL — интерактивная консоль

REPL (Read-Eval-Print Loop) — интерактивный режим Node.js. Вы вводите выражение, Node.js его вычисляет и выводит результат. Это удобно для быстрых экспериментов и отладки.

Чтобы войти в REPL, просто введите node без аргументов:

```
$ node
> 2 + 2
4
> 'Hello'.toUpperCase()
'HELLO'
> const arr = [1, 2, 3]
> arr.map(x => x * 2)
[ 2, 4, 6 ]
> Math.random()
0.7234...
> .help     // Список команд REPL
> .exit     // Выход из REPL
```

Полезные команды REPL (точка перед командой нужна):

.help — справка,
.clear — очистить контекст,
.editor — многострочный режим,
.exit — выход.

Также можно нажать Tab дважды для автодополнения.

1.5 Event Loop (событийный цикл)

Событийный цикл — это сердце Node.js. Понимание Event Loop критически важно для написания эффективного серверного кода.

Традиционные серверы (Apache, PHP) создают отдельный поток для каждого входящего запроса. Если сервер обрабатывает 10 000 одновременных подключений, ему нужно 10 000 потоков, каждый из которых потребляет память.

Node.js работает иначе: один поток обрабатывает все запросы. Когда приходит запрос, требующий чтения файла или обращения к базе данных, Node.js не ждёт завершения операции, а регистрирует callback-функцию и переходит к следующему запросу. Когда операция завершается, callback помещается в очередь, и Event Loop его выполняет.

Это называется неблокирующий ввод/вывод (Non-blocking I/O). Благодаря этому один процесс Node.js может обрабатывать десятки тысяч одновременных подключений.

Event Loop обрабатывает задачи в определённом порядке. Рассмотрим пример, создайте новый файл eventloop.js с содержимым:

```
console.log('1: Начало');


setTimeout(() => {
    console.log('2: Таймер (макрозадача)');
}, 0);


Promise.resolve().then(() => {
    console.log('3: Promise (микрозадача)');
});


console.log('4: Конец');
```

Запустите его выполнив

```
node eventloop.js
```

Вывод скрипта будет следующим

```
1: Начало
4: Конец
3: Promise (микрозадача)
2: Таймер (макрозадача)
```

Почему? Казалось бы, текст “таймер” должен быть выведен сразу после текста “Начало”, как в других скриптовых языках, например PHP.

А тут все немного по-другому. Порядок выполнения:

- Сначала выполняется весь синхронный код (строки 1 и 4)
- Затем микрозадачи — Promise.then(), queueMicrotask(). Они имеют высший приоритет
- Затем макрозадачи — setTimeout, setInterval, операции ввода/вывода

Это значит, что даже setTimeout с задержкой 0 мс выполнится после всех промисов. Понимание этого порядка поможет избежать неожиданных багов в асинхронном коде.

1.6 Глобальные объекты Node.js

В Node.js доступны несколько глобальных объектов, которых нет в браузере:

- __dirname — абсолютный путь к директории текущего файла (только в CommonJS)
- __filename — абсолютный путь к текущему файлу (только в CommonJS)
- global — глобальный объект (аналог window в браузере)
- Buffer — для работы с бинарными данными (изображения, файлы)
- setTimeout, setInterval, setImmediate — таймеры
- console — вывод в терминал (console.log, console.error, console.table, console.time)

Функция console.table() особенно полезна для отладки — она выводит массивы и объекты в виде таблицы. Создайте новый файл table.js с содержимым

```
const users = [
    { name: 'Алексей', age: 25 },
    { name: 'Мария', age: 30 },
];
console.table(users);
```

И затем выполните его, запустив

```
node table.js
```

Скрипт выведет таблицу

Функции console.time() и console.timeEnd() позволяют измерять время выполнения. Создайте новый файл loop.js с содержимым:

```
console.time('loop');

let sum = 0;
for (let i = 0; i < 1_000_000; i++) {
  sum += i;
}

console.timeEnd('loop');
```

И выполните его, запустив

```
node loop.js
```

Скрипт выведет время выполнения в миллисекундах

Практика

Задание 1.1:

- Установите Node.js через nvm, проверьте версию
- Создайте скрипт info.js, который выводит: дату, платформу, версию Node, аргументы командной строки, использование памяти
- Поэкспериментируйте с REPL: попробуйте работу с массивами, объектами, Math, Date
- Напишите пример с setTimeout, setImmediate и Promise — предскажите порядок вывода, затем проверьте
- Используйте console.time/timeEnd для измерения времени цикла из 1 000 000 итераций